Software Testing Documentation

Prepared by Om Satyawan Pathak
for the project APPLE WEBSITE AUTOMATION TESTING

LJKU - Department of Computer Applications

Software Testing Project 2025 - 2026

Om Satyawan Pathak

1. Introduction

1.1 Document Identifier

This document describes the testing strategy, test cases, execution process, and results for the Apple Website Automation Testing project.

The purpose of this project is to automate functional testing of the Apple website using Selenium WebDriver. The automation script validates important website functionalities such as:

- Website loading
- Navigation menu operation
- Search functionality
- Footer section expansion/collapse
- Page scrolling behavior
- Popup (if any) handling

This document provides detailed test scenarios and expected outcomes.

1.2 Scope

The purpose of this document is to define the test cases required to validate the functionality of the Apple website automation script.

The testing covers:

- Homepage accessibility
- Responsive viewport loading
- Search functionality
- Navigation menu operations
- Footer accordion sections
- Scrolling operations
- User authentication
- Error logging
- Browser closing

The document explains:

- Test objectives
- Test inputs
- Expected outputs
- Test environment
- Pass/fail criteria

1.3 References

References:

1. Selenium WebDriver Documentation
2. Python Selenium Framework Documentation
3. Apple Official Website
4. IEEE Standard for Software Testing Documentation

1.4 Testing Level in Overall Sequence

The following testing levels are considered:

Integration Testing

Different Selenium modules are tested together:

- Browser initialization
- Website loading
- Popup handling
- Menu interaction
- Search module
- User Authentication
- Footer interaction

System Testing

The complete automation workflow is executed to verify that the complete system behaves according to requirements.

Acceptance Testing

The automation verifies whether the website functions meet expected user interaction requirements.

1.5 Test Classes and Overall Test Conditions

The following test classes are included:

Test Class	Description
Functional Testing	Checks website features
UI Testing	Checks menu, buttons, sections
Navigation Testing	Checks page movement
Automation Testing	Validates Selenium execution
Logging Testing	Validates error recording

---

2. Details for System Test Plan

2.1 Features To Be Tested

The following features will be tested:

1. Apple website loading
2. Country popup closing
3. Sidebar menu opening/closing
4. Footer navigation sections
5. Search bar functionality
6. Multiple product searches
7. User Authentication
8. Page scrolling
9. Browser termination
10. Logging mechanism

2.2 Features Not To Be Tested

The following are outside the scope:

- Apple account authentication
- Payment systems
- Backend API testing
- Database testing
- Security penetration testing

2.3 Testing Approach

Black-box functional testing approach is used.

The tester verifies:

- Input given to website
- User action simulation
- Output behavior

Internal Apple website implementation is not considered.

Automation tool:

Selenium WebDriver
Python
Chrome Browser

2.4 Item Pass / Fail Criteria

A test case is considered:

PASS

When:

- Expected UI behavior occurs
- No Selenium exception occurs
- Correct output is obtained

FAIL

When:

- Element is missing
- Action cannot be performed
- Unexpected result occurs
- Selenium timeout occurs

2.5 Test Deliverables

Deliverables include:

- Selenium automation script
- Test case documentation
- Execution logs
- Test result report
- Test Documentation

---

3. Test Management

3.1 Planned Activities and Tasks

Testing process:

1. Analyze website requirements
2. Identify important user actions
3. Create automation scenarios
4. Execute Selenium scripts
5. Record logs
6. Analyze failures
7. Prepare final report

3.2 Environment / Infrastructure

Hardware Requirements

- Computer/Laptop
- Internet connection

Software Requirements

Operating System:

- Windows/Linux/Mac

Programming:

- Python 3.x

Libraries:

selenium
logging

time

Browser: Google Chrome

Automation: ChromeDriver

---

4. Test Case Details

4.1 Introduction

Each test case contains:

- Test Case ID
- Objective
- Input
- Expected Outcome
- Environment
- Dependencies

4.2 Test Cases

Test Case ID	Objective	Input	Expected Outcome	Environment	Dependencies	Result
TC01	Verify Apple website loads successfully	https://www.apple.com/	Apple Homepage loads successfully	Chrome Browser, Selenium WebDriver	None	PASS
TC02	Verify country/location popup	none	If popup appears, it should close	Chrome Brows	TC01	PASS

	handling (if any)		automatically.	er, Selenium WebDriver		
TC03	Verify sidebar menu functionality	Click menu button	Sidebar menu opens and closes successfully.	Chrome Browser	TC01, TC02	PASS
TC04	Verify footer scrolling and observe the accordions options.	Scroll to bottom of webpage , Click footer sections: ● Shop and Learn ● Apple Wallet ● Account ● Entertainment ● Apple Store ● Business ● Education ● Healthcare ● Government ● Apple Values ● About Apple	Page should move to footer section and each accordion must be clicked to verify its available options.	Selenium WebDriver	TC01, TC02	PASS
TC05	Verify search functionality.	Search for the following products: ● iPhone ● MacBook ● iPad ● Apple	Search results page should appear and the script must search for each of the mentioned products.	Chrome Browser	TC01, TC02	PASS

		Watch ● AirPods	Scroll down to the bottom and then return to top.			
TC06	Store every operations status in a log file (following logging mechanism)	Execute automation script.	Logs should record: ● Website opened ● Popup status ● Search execution ● Errors	Selenium WebDriver	All tests	PASS
TC07	User authentication (via email)	User email-id and password	Credentials gets entered	Chrome Browser	TC01	FAIL

NOTE: A CI/CD pipeline has been implemented using GitHub Actions to automatically execute the testing process on every push to the GitHub repository. This ensures continuous integration and automated validation of the application changes.

---

5. System Test Report Details

5.1 Overview of Test Results

Test Case	Description	Result
TC01	Website Loading	PASS

TC02	Popup Handling	PASS
TC03	Menu Testing	PASS
TC04	Footer Scroll and display options that are available in each accordion	PASS
TC05	Search Testing	PASS
TC06	Logging	PASS
TC07	User Authentication	FAIL

5.2 Detailed Test Results

All functional automation tests were executed successfully.

The Selenium script successfully:

- Opened Apple website
- Handled popup
- Tested navigation menu
- Tested footer sections
- Performed multiple searches
- Generated logs

5.3 Rationale For Decisions

Test cases were selected based on important user interactions.

The automation focuses on:

- High usage UI components
- Navigation workflows
- Search behavior
- Dynamic elements

5.4 Conclusions and Recommendations

The Apple website automation testing system successfully verifies important frontend functionalities.

Recommendations:

1. Add screenshot capture on failures.
2. Add pytest framework.
3. Add HTML test reports.
4. Implement Manual testing wherever necessary (like in this case: user authentication)
5. Add cross-browser testing.
